

CSCE 670 INFORMATION RETRIEVAL · MODULE 09 · LECTURE NOTES

Query Understanding

Working out what was meant.

MODULE OUTCOMES

- Explain spelling correction and its requirements.
- Explain query expansion and its risk.
- Explain relevance feedback, explicit and implicit.
- Explain query segmentation and intent classification.
- Build a query understanding pipeline and measure it.

1 Spelling correction

A MISPELLED QUERY RETURNS NOTHING USEFUL, AND A LARGE FRACTION OF QUERIES ARE MISPELLED

Measured misspelling rates in real query logs are substantial — well into the double digits for some domains — and a misspelled rare term matches no documents at all, so the failure is total rather than degraded, which makes this the highest-value preprocessing step in the course.

And the correction must use the collection and the query log rather than a dictionary — because the right correction depends on what people actually search for and what documents actually exist, which a dictionary has no information about. A dictionary will happily correct a correct product name into an English word.

So the standard approach is a noisy channel model: choose the candidate correction maximising $P(\text{intended}) \times P(\text{typed} | \text{intended})$, with the prior estimated from the query log and the error model from weighted edit distance — which handles keyboard adjacency and common substitutions.

And the interface matters at least as much as the model: correct silently when the model is confident, offer 'did you mean' when it is not, and always leave the original reachable — because a wrong confident correction is infuriating and loses more trust than the correction gains (CSCE 701 Module 11's usability argument).

2 Expansion

PSEUDO-RELEVANCE FEEDBACK

run the query, assume the top k results are relevant, extract their distinctive terms, add them to the query, and run it again. Needs no user input at all, and is the standard method.

THESAURUS AND EMBEDDING EXPANSION

add synonyms from a thesaurus, or the nearest neighbours of the query terms in an embedding space (CSCE 638 Module 04).

QUERY LOG EXPANSION

add terms from other queries that led to the same clicked documents. Frequently the best source, because it reflects actual expressed intent rather than semantic similarity.

AND THE DANGER IS DRIFT

if the top k results were about the wrong topic, the expansion amplifies the error -- and the second query is now confidently wrong. Which is why expansion helps on average and can hurt badly on individual queries.

S0: weight the expansion terms below the originals, and report the per-query distribution, not the mean.

Expansion helps on average and hurts badly on individual queries — which is exactly why Module 05 §4's per-query reporting matters, and is the cleanest concrete case for it in the course. A mean NDCG improvement from expansion routinely conceals a subset of queries made much worse, and those queries are the ones users complain about. **Query drift is the concrete argument for per-query reporting**, and it generalises to every averaged improvement.

3 Feedback

- **Explicit relevance feedback** — the user marks some results as relevant and the query is automatically reformulated. **It works very well and almost nobody will do it**, which is **a usability finding rather than a technical one** and is why the method is historically important and practically rare.
- **Rocchio's method is the classical formulation: move the query vector toward the centroid of the relevant documents and away from the centroid of the non-relevant ones**, with weights on each term — which is simple, effective, and interpretable.
- **Implicit feedback** — clicks, dwell time, query reformulations, and abandonment. **Abundant, biased, and the only kind you will actually get** at scale (Module 05 §4's position bias).
- **And session context is the strongest implicit signal: the previous query in a session usually disambiguates the current one** — 'jaguar' after 'car reviews' is unambiguous — which is cheap to use and frequently neglected.

- **Which makes reformulation detection particularly valuable — a reformulated query is a reported failure of the previous one**, delivered voluntarily. **A reformulation is a user telling you the last result was wrong**, which is **a free, strong, and very widely ignored quality signal** — and aggregating reformulations is one of the best available sources of failing queries for Module 05 §4's error analysis.

4 Structure and intent

- **Segmentation** — working out which adjacent words form a single unit. **'New York hotel' is two units and not three words**, and **treating it as three retrieves badly** because it will match documents about York and about newness.
- **Entity recognition and linking** — identifying that a span of the query names a specific known entity, **which enables structured retrieval against a knowledge base** alongside the text search and is how most commercial engines handle entity queries.
- **Intent classification** (Module 01 §3) — **which determines the result presentation at least as much as the ranking**: a navigational query wants one result, and a transactional one wants a filterable list.
- **Attribute extraction** — 'red shoes under £50' contains a colour, a category, and a price constraint, **which should become structured filters rather than being matched as text** (and matching '50' as text is a characteristic failure).
- **And none of it is free: each component adds latency and its own failure mode, so measure the end-to-end effect rather than the component's accuracy. A 95%-accurate intent classifier may hurt overall quality if the 5% are confidently misrouted** to the wrong presentation, which is Module 05's measurement discipline applied to a pipeline stage.

AND A LANGUAGE MODEL CHANGES WHAT IS AVAILABLE HERE

Query rewriting, expansion, decomposition, and reformulation are all things a language model does well — and it can resolve a pronoun against the session history, decompose a compound question into retrievable parts, or generate a hypothetical answer document to retrieve against, none of which the classical methods do.

Which is a real improvement over the classical methods for conversational and genuinely complex queries, and is the query-understanding half of Module 11's architecture.

At a cost that is easy to understate: a model call before retrieval adds latency to every single query and introduces a failure mode that is hard to bound — a rewrite that changes the meaning is worse than no rewrite (Module 04 §4's budget, and §2's drift in a new form).

So the honest position: use it where the query is genuinely hard, and not on the head of the distribution — where a cached classical pipeline is both faster and considerably more predictable (Module 04 §4's caching). **Routing by query difficulty is the design that gets both**, and it requires a difficulty estimate, which is itself a query understanding problem.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Manning, Raghavan & Schütze, chapters 3 and 9 (free)	§1's correction and §2 and §3's expansion and feedback , with Rocchio derived.
Croft, Metzler & Strohman, chapter 6 (free PDF)	§4's query understanding — segmentation, intent, and the production pipeline shape.
Cucerzan & Brill — Spelling correction as an iterative process (free)	§1's log-based approach — why the query log beats a dictionary, with the method.
Gao et al. — Precise Zero-Shot Dense Retrieval without Relevance Labels (free)	§4's callout — generating a hypothetical document to retrieve against, which is the clearest instance of the shift.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Measure the misspelling rate** in any query log you can obtain.
- 2 **Show that a misspelled rare term returns nothing** on your collection.
- 3 **Build a noisy channel corrector** with a log-derived prior.
- 4 **Find a case where a dictionary corrector is wrong** and a log-based one is right.
- 5 **Implement pseudo-relevance feedback** and measure the mean improvement.
- 6 **Plot the per-query change** and count the queries made worse.
- 7 **Find a drifted query** and trace which expansion term caused it.
- 8 **Implement Rocchio** and test it with judgements as explicit feedback.
- 9 **Detect reformulations** in a session log and inspect the queries that preceded them.
- 10 **Add an intent classifier** and measure the end-to-end effect rather than its accuracy.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is a misspelled query a total rather than partial failure?
- 2 Why must correction use the log rather than a dictionary?
- 3 Give the noisy channel formulation and the interface rules.
- 4 Name three expansion methods and the danger they share.
- 5 Why does expansion require per-query reporting?
- 6 Contrast explicit and implicit feedback, and say why one is rare.

- 7 Describe Rocchio's method.
- 8 Why is a reformulation a valuable signal?
- 9 Name four things query understanding extracts.
- 10 What should you measure about a query understanding component?